

Apuntes MongoDB / NoSQL

Diplomado Data Science — TEC de Monterrey / The Learning Gate

1. Estructura de una consulta

- Una **consulta** (query) en MongoDB es un documento JSON. Es el equivalente al `SELECT` de SQL.
- Las consultas tienen estructuras y palabras reservadas específicas (operadores con `$`).
- Se hacen desde Compass (campo **Filter**) o desde Python (PyMongo).

Consulta más básica

```
js
{ }
```

Documento vacío → regresa **todos** los documentos de la colección.

Consulta por un campo

```
js
{ "alcohol": 9.4 }
```

Busca todos los documentos donde el campo `alcohol` valga `9.4`.

Los saltos de línea son opcionales: `{"alcohol":9.4}` es válido también.

Pasos en Compass

1. Seleccionar la colección.
2. Escribir la consulta en el campo **Filter**.
3. Clic en **Find** → Compass muestra la lista de documentos que coinciden y el total encontrado.

Consultas con múltiples campos (AND implícito)

```
js
```

```
{ "alcohol": 9.4, "quality": 4 }
```

Regresa documentos que cumplan **ambas** condiciones a la vez.

Detalles importantes

- MongoDB busca **coincidencia exacta** de valor. En campos de texto (string), **distingue mayúsculas/minúsculas**.
- Si **no existe** ningún documento con el campo/valor buscado → MongoDB regresa una **lista vacía** (en Compass: *"No results"*). A diferencia de SQL, no es un error.
- MongoDB **no obliga una estructura fija** por colección: puedes tener documentos con campos y tipos distintos dentro de la misma colección (esto es clave para entender ventajas/desventajas de NoSQL: flexibilidad vs. consultas potencialmente menos eficientes si la info es muy heterogénea).

2. Proyecciones

Por default, una consulta regresa **todos los campos** de los documentos encontrados. Cuando solo necesitas algunos campos, usas una **proyección**.

- En Compass: botón **"Options"** → aparece el campo **Project**.

Tipos de proyección

Tipo	Cómo se hace	Qué pasa
Inclusión	Asignar <code>1</code> o <code>true</code> a los campos	Solo se muestran esos campos (+ <code>_id</code> por default)
Exclusión	Asignar <code>0</code> o <code>false</code> a los campos	Se muestran todos los campos excepto esos

Ejemplo (inclusión)

Para la consulta `{"reviewerName": {"$regex": "/dave/"}}`, si quieres mostrar solo `reviewerName`, `overall` y `summary`:

```
js
{ "reviewerName": 1, "overall": 1, "summary": 1 }
```

El resultado incluirá esos 3 campos + `_id` (siempre se muestra a menos que lo excluyas).

Ejemplo (exclusión)

```
js
{ "reviewerName": 0, "overall": 0, "summary": 0 }
```

Muestra **todos los campos excepto** esos tres.

Reglas importantes

-  **No puedes mezclar inclusión y exclusión** en el mismo documento de proyección.
- El **único campo que puede llevar 0 dentro de una proyección por inclusión** es `_id` (es la excepción a la regla anterior).

Ocultar `_id` en una proyección por inclusión

```
js
{ "reviewerName": 1, "overall": 1, "summary": 1, "_id": 0 }
```

Glosario

- **Proyección:** define qué campos se incluyen/excluyen en el resultado de una consulta.
 - **Proyección por inclusión:** campos con `1`/`true`.
 - **Proyección por exclusión:** campos con `0`/`false`.
-

3. Agregaciones — Etapas (pipeline)

Las **agregaciones** permiten transformar, filtrar y resumir datos mediante una estructura de **etapas encadenadas** (pipeline). La salida de una etapa es la entrada de la siguiente ("tubería").

¿Para qué sirven las agregaciones?

- Agrupar valores de múltiples documentos.
- Ejecutar operaciones sobre datos agrupados (total, promedio, mín, máx, etc.).
- Analizar cambios en el tiempo.

Formato general

```
js  
[ etapa1, etapa2, etapa3 ]
```

Es un **arreglo** donde cada etapa es un documento BSON bien formado.

En Compass

- Seleccionar colección → pestaña "**Aggregations**".
- Botón "+ **Add Stage**" para agregar etapas.
- Compass solo te pide el contenido interno de la etapa (el resto del documento lo agrega automáticamente).
- El texto entre `/** ... */` que agrega Compass son **comentarios**, no afectan la ejecución; se pueden borrar.
- La vista previa solo muestra una **muestra de 20 documentos** (configurable) — no es el total real de resultados.

Etapas más comunes

Etapas	Descripción
<code>\$match</code>	Filtra documentos que cumplan una condición (usa operadores normales de consulta)
<code>\$group</code>	Agrupar por un campo/identificador y aplica acumuladores
<code>\$count</code>	Cuenta documentos y guarda el total en un campo con el nombre que tú definas
<code>\$project</code>	Pasa solo los campos especificados (igual que proyección, pero dentro de un pipeline)
<code>\$sort</code>	Ordena: <code>1</code> ascendente, <code>-1</code> descendente
<code>\$limit</code>	Pasa solo los primeros <i>n</i> documentos
<code>\$addFields</code>	Agrega un campo nuevo calculado a cada documento (alias: <code>\$set</code>)
<code>\$out</code>	Escribe el resultado en una colección nueva, reemplazándola si ya existe. Debe ser la última etapa
<code>\$merge</code>	Escribe el resultado en una colección, pero puede agregar/sobrescribir documentos sin reemplazar toda la colección. Debe ser la última etapa

Ejemplo: `$match` + `$count`

Etapla 1 — filtrar mujeres:

```
js
{ "$match": { "gender": "female" } }
```

Etapla 2 — contar resultado:

```
js
{ "$count": "total_female_laureates" }
```

`$count` siempre regresa **un solo documento** con el campo que nombraste. (Resultado del ejemplo del curso: 53 mujeres premiadas con el Nobel.)

Cambiando la condición a `{"death": {"$exists": 0}}` se obtienen los **premiados vivos** (317 en el ejemplo); con `{"$exists": 1}` los **fallecidos** (626).

`$group`: sintaxis

```
js
{
  "$group": {
    "_id": <expression>,           // campo(s) por los que agrupas
    "<campo1>": { "<acumulador1>": <expr1> },
    ...
  }
}
```

⚠ **Regla clave:** cuando un nombre de campo aparece del **lado derecho** de los `:`, debe llevar el prefijo `$` (significa "el valor de ese campo").

Ejemplo — total de premiados por país:

```
js
{
  "$group": {
    "_id": "$birth.place.country.en",
    "total_laureates": { "$count": {} }
  }
}
```

`$count` aquí es un **operador de agregación** dentro de `$group` (no confundir con la **etapa** `$count`, que es independiente).

`$sort`

```
js
{ "$sort": { "total_laureates": -1 } }
```

Tabla de operadores de agregación más usados

Operador	Tipo	Descripción
<code>\$abs</code>	Aritmético	Valor absoluto
<code>\$add</code>	Aritmético	Suma (números o número+fecha)
<code>\$subtract</code>	Aritmético	Resta
<code>\$multiply</code>	Aritmético	Multiplica
<code>\$divide</code>	Aritmético	Divide
<code>\$and</code> / <code>\$or</code> / <code>\$not</code>	Booleano	Lógicos estándar
<code>\$size</code>	Arreglo	Número de elementos de un arreglo
<code>\$isNumber</code>	Tipo de dato	<code>true</code> si la expresión es numérica
<code>\$avg</code>	Acumulación	Promedio (ignora valores no numéricos)
<code>\$count</code>	Acumulación	Cuenta documentos en un grupo
<code>\$first</code> / <code>\$last</code>	Acumulación	Primer/último doc de un grupo
<code>\$max</code> / <code>\$min</code>	Acumulación	Valor máximo/mínimo
<code>\$sum</code>	Acumulación	Suma (ignora no numéricos)
<code>\$concat</code>	String	Concatena cadenas
<code>\$toLowerCase</code> / <code>\$toUpperCase</code>	String	Cambia mayúsc./minúsc.
<code>\$ifNull</code>	Condicional	Si no es nulo regresa su valor; si sí, regresa el segundo argumento

Operador	Tipo	Descripción
<code>\$cond</code>	Condicional	Ternario: <code>if / then / else</code>

Cada **tipo de etapa** acepta solo ciertos operadores (ej. `$group` usa los de tipo *Acumulación*).

Ejemplo combinado: clasificar vivos/muertos en un solo pipeline

Etapa 1:

```
js
{
  "$addField": {
    "live": {
      "$cond": {
        "if": { "$ifNull": ["$death", false] },
        "then": "dead",
        "else": "live"
      }
    }
  }
}
```

Etapa 2:

```
js
{
  "$group": {
    "_id": "$live",
    "total_laureates": { "$sum": 1 }
  }
}
```

(Aquí `$sum: 1` suma "1" por cada documento del grupo = forma clásica de contar dentro de `$group`.)

Ejemplo: año con más premiados (pipeline de 4 etapas)

```
js
```

```
[
  { "$match": { "laureates": { "$exists": 1 } } },
  {
    "$group": {
      "_id": "$awardYear",
      "total_laureates": { "$sum": { "$size": "$laureates" } }
    }
  },
  { "$sort": { "total_laureates": -1 } },
  { "$limit": 1 }
]
```

Interpretación paso a paso:

1. Solo años donde sí hubo premiados.
2. Agrupa por año y suma el tamaño del arreglo `laureates` (cuántas personas/organizaciones ganaron ese año).
3. Ordena de mayor a menor.
4. Se queda solo con el primer documento (el año con más premiados).

Glosario

- **Agregación:** proceso para agrupar/transformar/analizar datos de múltiples documentos.
- **Campo anidado:** campo dentro de un objeto, ej. `birth.place.country.en`.
- **Etapas (\$stage):** cada bloque del pipeline.
- **Pipeline:** arreglo secuencial de etapas.
- **Operadores de acumulación:** usados dentro de `$group` (`$sum`, `$avg`, `$min`, `$max`...).

4. Uso de `$lookup` (joins entre colecciones)

Sirve para **relacionar** dos colecciones de la misma base de datos sin duplicar información (similar a un `JOIN` en SQL).

¿Cómo funciona?

`$lookup` toma dos colecciones y **agrega un campo tipo arreglo** a los documentos de la primera colección, con los documentos de la segunda colección que cumplen la condición de unión.

Etapa `$unwind` (paso previo común)

Cuando un campo es un arreglo (ej. `laureates` dentro de `awards`), donde cada award puede tener varias personas ganadoras), `$unwind` separa el arreglo en **un documento por cada elemento**:

```
js
{ "$unwind": { "path": "$laureates" } }
```

Ejemplo completo: simplificar y guardar en nueva colección

Etapa 1 — separar arreglo:

```
js
{ "$unwind": { "path": "$laureates" } }
```

Etapa 2 — proyectar y renombrar campos (`$project` permite renombrar):

```
js
{
  "$project": {
    "name": "$laureates.knownName.en",
    "awardYear": 1,
    "category": "$category.en",
    "_id": 0
  }
}
```

Etapa 3 — guardar resultado en colección nueva:

```
js
{ "$out": "awards_simple" }
```

Al correr el pipeline, Compass avisa *"A write operation will occur"* → confirmar con **"Yes, run pipeline"**.

Etapa `$lookup`: sintaxis

```
js
```

```
{
  "$lookup": {
    "from": "laureates",
    "localField": "name",
    "foreignField": "knownName.en",
    "as": "laureate_detail"
  }
}
```

Lectura: "Estando en `awards_simple`, agrega a cada documento un campo `laureate_detail` con los documentos de `Laureates` donde `knownName.en` coincide con el campo `name` del documento actual."

Glosario

- **Campo común:** atributo compartido entre dos colecciones (como una llave foránea).
- **Join (unión):** combinar datos de dos fuentes según un campo común → en MongoDB se logra con `$lookup`.
- **Relación:** vínculo lógico basado en un atributo compartido.

5. Ejercicios resueltos — "Identificación de poblaciones" (caso Premio Nobel)

Caso de estudio: la Fundación Nobel quiere analizar representación (género, país) en la historia del premio. Dataset: **Nobel Prize Dataset** (Kaggle), dos colecciones: `laureates` (personas) y `awards` (premios por año/categoría).

Misión 1 — Preparar entorno y datos

Cadena de conexión válida (MongoDB Atlas):

```
mongodb+srv://<db_username>:<db_password>@cluster0.otxeh4d.mongodb.net/?
retryWrites=true&w=majority&appName=Cluster0
```

- ✓ Debe incluir usuario/contraseña (reemplazándolos por los reales) y el clúster correcto.
- ✗ Nunca usar `localhost` para un servicio en la nube.

Conectarse desde Python con PyMongo:

```
python
```

```
from pymongo import MongoClient
client = MongoClient(<cadena_de_conexión>)
```

Para un servidor **local**, simplemente: `client = MongoClient()` (sin parámetros).

Referenciar base de datos y colección (notación de propiedad):

```
python

nobel = client.nobel
laureates = nobel.laureates
```

Misión 2 — Consultando y explorando

Guardar resultados de `find()` como listas de Python (recuerda: `find()` regresa un `Cursor`), hay que convertirlo con `list()`):

```
python

laureates_female = list(laureates.find({"gender": "female"}))
laureates_male = list(laureates.find({"gender": "male"}))
```

Contar mujeres ganadoras:

```
python

nobel.laureates.count_documents({"gender": "female"})
```

Contar laureados nacidos en México (el campo correcto es anidado):

```
python

nobel.laureates.count_documents({"birth.place.countryNow.en": "Mexico"})
```

Misión 3 — Agregaciones

Total de premios por género (pipeline correcto):

```
python
```

```

pipeline_gender = [
    {
        '$match': { 'gender': { '$exists': 1 } }
    }, {
        '$group': {
            '_id': '$gender',
            'total_laureates': { '$sum': 1 }
        }
    }
]

laureates_by_gender = list(laureates.aggregate(pipeline_gender))

```

⚠ Errores comunes: usar sintaxis SQL directamente (no es compatible), o usar `'field'` en vez de `'_id'` dentro de `$group` (el identificador del grupo **siempre se llama** `_id`).

Proyección para mostrar solo nombre y país (ocultando `_id`):

```

python

pipeline = [
    { "$project": { "_id": 0, "knowName.es": 1, "birth.place.country.en": 1 } }
]

```

(Recuerda: `0` = ocultar, `1` = mostrar; `_id` se muestra por default así que hay que ocultarlo explícitamente.)

Misión 4 — Automatizando con Python

Conexión local correcta:

```

python

from pymongo import MongoClient
client = MongoClient()

```

Filtrar laureados con más de un Nobel (campo `nobelPrizes` es un arreglo):

```

python

nobel.laureates.find({ "nobelPrizes": { "$size": 2 } })

```

Importante: el operador `$size` en una consulta normal (`find`) **no admite comparaciones como** `$gt` directamente. Si necesitas "más de 1" en general (no exactamente 2), se requiere una **agregación** en vez de un `find()` simple.

Notas finales / reflexiones del curso (para repasar)

- **Flexibilidad de esquema** en MongoDB: ventaja para datos heterogéneos, pero puede complicar consultas si una colección mezcla estructuras muy distintas → considerar separar en varias colecciones según el caso.
- **Proyecciones vs. colecciones separadas**: las proyecciones evitan tener que normalizar todo en distintas colecciones; ventaja en simplicidad, desventaja si el documento es muy pesado y casi nunca usas la mayoría de sus campos.
- **Agregaciones** son la herramienta más poderosa para convertir datos "crudos" en respuestas de negocio (tendencias, totales, comparaciones).
- **\$lookup** evita duplicar información, relacionando colecciones bajo un campo común — el equivalente más cercano a un JOIN de SQL en MongoDB.